

The OpenFlow Protocol and SDN Interaction

1 The OpenFlow Protocol

The **OpenFlow protocol** operates between an SDN controller and an SDN-controlled switch (or other devices implementing the OpenFlow API). It is the most widely adopted protocol for the "southbound" interface.

Protocol Basics

- **Transport:** Operates over **TCP**.
- **Port:** Default port number is **6653**.

1.1 Controller-to-Switch Messages

These messages allow the controller to manage the underlying data plane:

- **Configuration:** Allows the controller to query and set configuration parameters.
- **Modify-State:** Used to add, delete, or modify entries in the switch's **flow table** and set port properties.
- **Read-State:** Used to collect statistics and counter values from the flow table and ports.
- **Send-Packet:** Allows the controller to send a specific packet out of a specified port at the switch.

1.2 Switch-to-Controller Messages

These messages allow switches to inform the controller of network events:

- **Flow-Removed:** Informs the controller that a flow table entry has been removed (e.g., due to a timeout).
- **Port-Status:** Informs the controller of a change in port status (e.g., a link going up or down).
- **Packet-in:** Sends a packet to the controller for additional processing if it doesn't match any flow table entry (or if explicit "send to controller" action is matched).

2 Data and Control Plane Interaction: A Case Study

To understand how these components work together, consider a shortest-path routing scenario using Dijkstra's algorithm.

Key Differences from Traditional Routing

- **Decentralized Logic:** Dijkstra's algorithm runs as an external application, not inside the switches.
- **Direct Notification:** Switches send link updates directly to the controller instead of flooding them to each other.

2.1 Example: Link-State Change Workflow

Assume a link between switch s_1 and s_2 goes down. The following 6 steps occur:

1. **Detection:** Switch s_1 experiences a link failure and notifies the controller using an OpenFlow **port-status** message.
2. **Update:** The controller receives the message and notifies the link-state manager, which updates the network graph/database.
3. **App Notification:** The **Routing Application** (which registered for such events) is notified of the link-state change.
4. **Recalculation:** The routing application consults the state-management layer and computes new least-cost paths.
5. **Table Determination:** The routing application interacts with the **flow table manager** to determine which switch tables need updates.
6. **Implementation:** The flow table manager uses OpenFlow (Modify-State messages) to update the flow tables at all affected switches (e.g., s_1 , s_2 , and s_4).

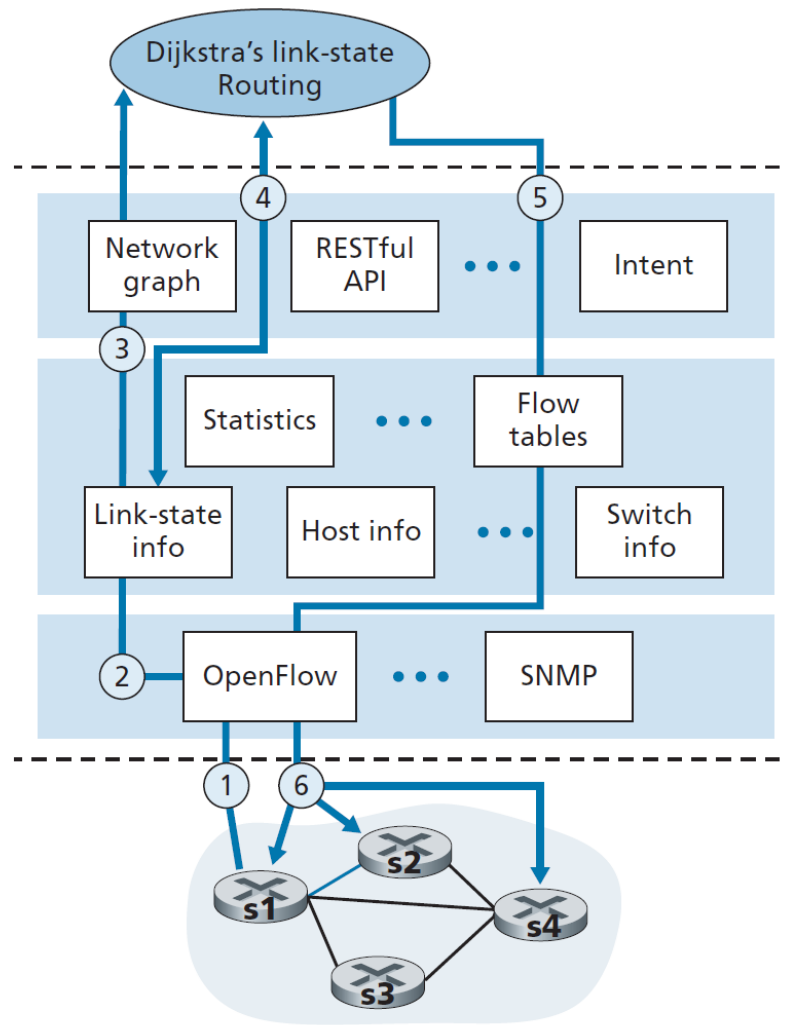


Figure 1: SDN Interaction Scenario: Updating flow tables after a link failure.

The Power of SDN

This ease of changing network behavior—simply by updating application software—contrasts sharply with traditional per-router control, where software in every router (often from different vendors) would need to be coordinated or changed.